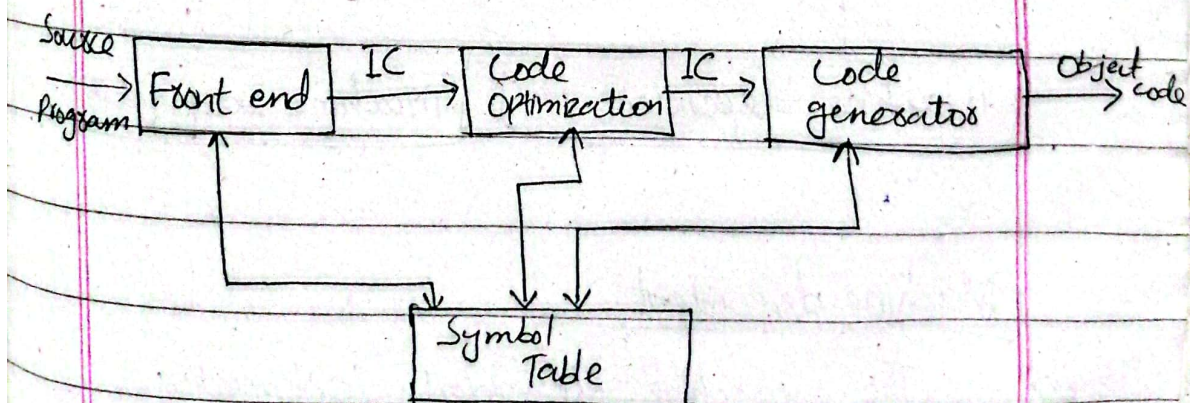


Ch#

Code Generation and Optimization

- ⇒ Code generation is the final phase of code in the process of compilation.
- ⇒ it takes intermediate code as input and generate target machine code as output.

Position of Code generators in Compilation Process:

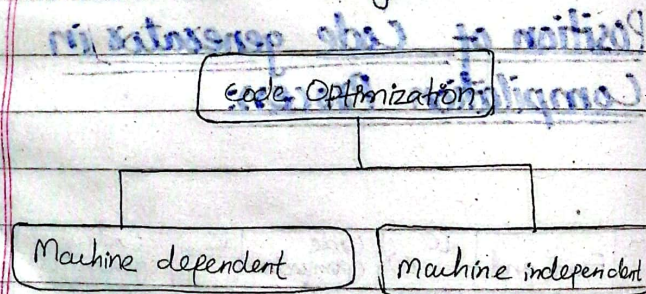


Date: _____

Day: MTWTF

Code Optimization

- Code Optimization is the fifth phase of compiler.
- Code Optimization is required to produce an efficient code.
- The improvement over Intermediate code by transformation is called optimization.
- The classification of optimization is done in two categories:



Machine dependent:

- ⇒ The machine dependent optimization is based on characteristics of the target machine code for the instruction

Date: _____

Day: MTWTF

set used and addressing modes used for the instructions to produce the efficient target code. & register allocation

- ⇒ The machine dependent optimization can be achieved using following criteria:

- Allocation of sufficient number of resources to improve the execution efficiency of the program.
- Using intermediate instructions whenever necessary.
- The use of intermediate instructions.
- The interleaving of instruction along with the data increase the speed of execution.

Machine independent:

- ⇒ The machine independent optimization is based on the characteristics of the programming languages for appropriate programming structure and usage of efficient arithmetic properties involved to

Date: _____

Day: MTWTF

reduce the execution time.
⇒ The machine independent optimization can be achieved using following criteria:

- The code should be analyzed completely and use alternative equivalent sequence of source code that will produce a minimum amount of target code.
- Use appropriate program structure in order to improve the efficiency of target code.
- From the source code eliminate the unreachable code.
- Move two or more identical computations at one place and make use of the result instead of each time computing the expression.

Local optimization Global optimization
- interprocedural optimization Loop optimization

Date: _____

Day: MTWTF

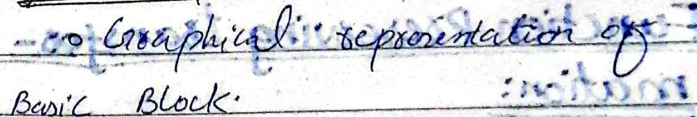
Criteria that must be satisfied:

- must preserve meaning of program.
- speedup program by meaning of program.
- must worth the effort

Terms used in code optimization

Basic Blocks:

- Break intermediate code into blocks. a basic block is a straight line code sequence with no branches in except to the entry & no branches out

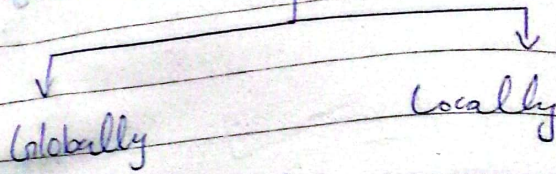
Graphical representation of Basic Block:  out except at the exit.

Key points:

- Process of code optimization involves:
 - Eliminating the unwanted lines of code
 - Rearranging the statements of the code.

Date: _____

Code Optimization Techniques



Globally vs Locally

- o If the transformation is applied on the same basic block then that kind of transformation is done locally.
- o Otherwise transformation is done globally. Generally the local transformation is done first.

1. Function Preserving transformation:

- o Common Sub expression elimination
- o Copy propagation
- o Dead code elimination
- o Loop Optimization
- o Constant folding

Date: _____

2. Loop Optimization:

- o Code motion
- o Induction variable
- o Reduction in strength.

Compile time evaluation

Also known as local optimization techniques.

⇒ Compile time evaluation means shifting of computations from run time to compilation time.

⇒ There are two methods to achieve compile time evaluation.

(1) Folding

- o In folding technique, the computation of constant is done at compile time instead of execution time.

for example: 3×2
length = 6

Here, folding is implied by performing the computation

Date: _____

Day: MTWTF

of 22/7 at compile time instead of execution time.
 Connect Issue $\rightarrow$ false

2. Constant Propagation:

$\Rightarrow$ In this technique the value of variable is replaced and computation of an expression is done at the compilation time.

for example:

$$\begin{array}{l} \pi = 3.14 \\ x = 5 \\ \text{Area} = \pi \times x \times x \end{array}$$

$$\begin{array}{l} b = 3 \\ c = 1 + b \\ d = b + c \end{array} \rightarrow \begin{array}{l} b = 3 \\ c = 4 \\ d = 3 + c \end{array}$$

$\Rightarrow$ Here at the compilation time the value of π is replaced by 3.14 and x by 5 then computation of $3.14 \times 5 \times 5$ is done during compilation.

Common SubExpression Elimination

Date: _____

Day: MTWTF

$\Rightarrow$ The common sub expression is an expression appearing repeatedly in the program which is computed previously.

$\Rightarrow$ Then if the operand of this ^{sub} expression do not get changed at all then result of such sub-expression is used instead of recomputing it each time.

for example:

$$\begin{array}{l} t_1 = i \times i \\ t_2 = a[t_1] \\ t_3 = 4 \times j \\ t_4 = i \times i \\ t_5 = n \\ t_6 = b[t_4] + t_5 \end{array}$$

The code can be optimized using the common sub-expression elimination as:

Date: _____

Day: M T W T F S

$$t_1 = 4 * i$$

$$t_2 = a[t_1]$$

$$t_3 = 4 * j$$

$$t_5 = n$$

$$t_6 = b[t_1] + t_5$$

⇒ The common sub expression $t_4 = 4 * i$ is eliminated as its computation is already in t_1 and value of i is not been changed from definition to use.

Variable Propagation / Copy Propagation

⇒ Variable propagation means use of one variable instead of another.
for example:

$x = p_i$ $area = x * 8 * 8$
--

$$x[i] = a$$

$$sum = x[i] * 8 + a; \rightarrow$$

$$= a * 8 + a$$

⇒ The optimization using variable propagation can be done as follows:

Date: _____

Day: M T W T F S

$$area = p_i * 8 * 8;$$

⇒ Here the variable x is eliminated.
o Necessary condition is that a variable must be assigned to another variable or some constant.

Code movement

⇒ There are two basic goals of code movement:

o To reduce the size of the code i.e. to obtain the space complexity.

o To reduce the frequency of execution of code i.e. to obtain the time complexity.

for example:

```
for (i=0; i<=10; i++)
{
```

$$x = 4 * 5$$

.....

$$k = (4 * 5) + 50;$$

}

Date: _____

Day: MTWTF

This can be optimized:

```
temp = y * z
```

```
for (i = 0; i <= 10; i++)
```

```
{
```

```
    k = z
```

```
    ...
```

```
    k = z + 50
```

```
}
```

⇒ The code for $y \times 5$ will be generated only once. And simply the result of that computation is used wherever necessary.

Loop Invariant Computation:

- The loop invariant optimization can be obtained by moving some amount of code outside the loop and placing it just before entering in the loop.
- This method is also called code motion.

Date: _____

Day: MTWTF

for example:

```
while (i <= MAX - 1)
```

```
{
```

```
    s = s + a[i];
```

```
}
```

⇒ The code can be optimized by removing the computation of $\text{MAX} - 1$ outside the loop. The optimized code can be:

```
n = MAX - 1
```

```
while (i <= n)
```

```
{
```

```
    s = s + a[i];
```

```
}
```

Strength Reduction

The strength of certain operators is higher than others.

For instance strength of $*$ is higher than $+$.

In strength reduction technique the higher strength operators can be replaced by lower

Replace expensive operations with simpler ones

Date: _____

Day: MTWTF

Strength operators.

```
for (i=1; i<=50; i++)
{
```

```
    count = i*7;
```

```
    ....
```

```
}
```

The count values are 7, 14, 21 and so on upto less than 50.

This code can be replaced by using strength reduction as follows:-

```
temp = 7;
```

```
for (i=1; i<=50; i++)
```

```
{
```

```
    ....
```

```
    count = temp;
```

```
    temp = temp + 7;
```

```
}
```

Date: _____

Day: MTWTF

⇒ The induction variable is integer scalar identifies used in the form of

$V = V \pm \text{constant}$, Here V is an induction variable.

⇒ The strength reduction is not applied to the floating point expressions because such a use may yield different rules.

Dead code elimination

A variable is said to be line in a program if the value contained into it is used subsequently.

On the other hand, the variable is said to be dead at a point in a program if the value contained into it is never been used.

Date: _____

⇒ The code containing such a variable supposed to be a dead code.

⇒ And an optimization can be performed by eliminating such a dead code.

for example:

$i = j;$

.....

$x = i + j;$

.....

⇒ The optimization can be performed by eliminating the assignment statement $i = j$.

⇒ This assignment statement is called dead assignment.

Another example:

$i = 0$

$\text{if } (i = 1)$

$\{$
 $x = x + 5;$

$\}$

Day: (M T W T F S S)

Date: _____

⇒ Here ~~if statement~~ is dead code as this condition will never get satisfied hence, if statement can be eliminated and optimization can be done.

No control flows into a basic block.

An assignment is dead. e.g. assign a value to a dead variable.

$x = y + 1$

$y = 5$

$x = 2 * 2$

$y = 5$

→ $x = 2 * 2$

$\text{if } (\text{false})$
 $\{$

$x = 5$

$\}$

→ $\text{if } (\text{false})$
 $\{$

$\}$

Date: _____

Day: MTWTFSS

Loop Optimization

- Problem of increasing execution speed and reducing the overheads associated with loops.

⇒ Loop optimization is a technique in which code optimization is performed on inner loops.

⇒ The loop optimization is carried out by the following methods:

- (1) Code motion
- (2) Induction variable and strength reduction
- (3) Loop invariant method
- (4) Loop unrolling
- (5) Loop fusion/jamming

Code motion:

⇒ Techniques in which moves the code outside the loop.

⇒ if there lies some expressions in the loop whose result remains unchanged even after executing the loop for several times, then

Date: _____

Day: MTWTFSS

such an expression should be placed just before the loop.

for example:

```
while (i <= max-1)
{
    sum = sum + a[i];
}
```

After Optimization:

```
n = max-1
while (i <= n)
{
    sum = sum + a[i];
}
```

2) Induction variables and reduction in strength:

- used to replace variable from inner loop.

A variable x is called an induction variable of loop L if the value of variable

gets changed every time.
it is either decremented by
some constant or incremented by
some constant.

for example:

$i = i + 1$

$t_1 = 4 * i$

$t_2 = a[t_1]$

if $t_2 \leq 10$ goto B₁

⇒ The value of i and t_1 are locked
state.

⇒ Then, value of i gets changed or
incremented by 1 then t_1 gets
incremented by 4.

⇒ Hence i and t_1 are induction
variables.

⇒ When there are two or more induction
variables, it may be possible to get
rid of all but one.

Loop invariant:

⇒ In this technique the computation
inside the loop is avoided and
only the computation overhead
or complex is avoided. This
ultimately optimizes code generation.

for example:

```
for i = 0 to 10  
    k = i + a[i]  
    ----  
    ----  
end;
```

After optimization:-

```
t = a[0]  
for i = 0 to 10  
    k = i + t;
```


Date: _____

Loop unrolling

⇒ In this method the number of jumps and tests can be reduced by writing the code two times.

⇒ It reduces execution time but increases memory load time.

for example:

Before

```
int i=1
```

```
while (i <= 100)
```

```
{
```

```
    a[i] = b[i];
```

```
    i++;
```

```
}
```

After loop unrolling

```
int i=1
```

```
while (i <= 100)
```

```
{
```

```
    a[i] = b[i]
```

```
    i++;
```

```
    a[i] = b[i]
```

```
    i++;
```

```
}
```

Date: _____

Loop fusion:

o Also known as loop jamming.

In loop fusion method several

o loops are merged into one

loops. When two loops would iterate the same no of times, then their bodies for example: can be merged.

Code:

```
for i=1 to n
```

```
for i=1 to m
```

```
    a[i,j] = 10
```

After

```
for i=1 to n*m
```

```
    a[i] = 10
```

Example 2

```
for (i=0; i<5; i++)
```

```
    a = i+5
```

```
for (i=0; i<5; i++)
```

```
    b = i+10
```

After

```
for (i=0; i<5; i++)
```

```
{
```

```
    a = i+5
```

```
    b = i+10
```

```
}
```


Q. Construct the basic blocks and data flow graph and identify loop invariant statements?

```

for (i = 1 to n)
{

```

```

    j = 1
    while (j <= n)
    {

```

```

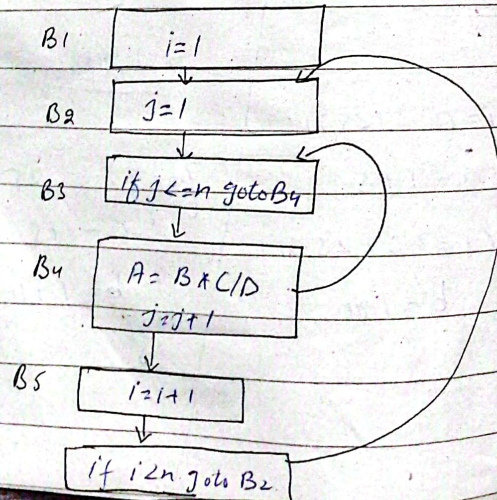
        A = B * C / D

```

```

        j = j + 1
    }
}

```



⇒ The loop invariant statement is:
 $A = B * C / D$

Optimized as:

```

t = C / D

```

```

while (j <= n)
{

```

```

    A = B * t

```

```

    j = j + 1
}

```


Questions

Q. What is the need of code optimization?

- o To produce the efficient target code.

Q. What are the criteria that needs to be considered while applying the code optimization techniques?

⇒ The semantic or meaning of the program must not be changed.

⇒ The improvement over the program efficiency must be achieved without changing the algorithm of the program.

Q. What is an optimizing compiler?

- o The optimizing compiler is a compiler that takes minimum amount of time to execute the program and also takes less amount of memory for executing the program.

Q. What is basic block?

- o A basic block is a sequence of consecutive statements in which the flow of control enters at the beginning and leaves at the end without branching or halt.

Q. Write algorithm to convert three address code to basic blocks?

Date: _____

Day: MTWTF

- 1) Determine the leader of the three-address code.

Method to determine leader:

(a) first statement of three-address code is a leader.

(b) Target statement of unconditional / conditional goto is a "Leader".

(c) Statement immediately following unconditionally / conditionally goto is a leader.

- 2) The Basic Block blocks starts at one "Leader" and ends just before the next "Leader".

Q. What is flow Graph?

⇒ A flow graph is a directed graph whose nodes are the basic blocks and edges are used to add the flow of information from one block to another.

Q. Optimize the following code?

```
void f(int i)
{
    a[0] = i + 0;
    a[1] = i * 0;
    a[2] = i - i;
    a[3] = 1 + i + 1;
}
```

After optimization:

```
void f(int i)
{
    a[0] = i;
    a[1] = 0;
    a[2] = 0;
    a[3] = 2 + i;
}
```

Q. Optimize code.

```
if (false)
{
    a = 5;
}
```

```
if (false)
{
}
```


Date: _____

Day (M T W T F S)

Q. Optimize the code:

Code Before optimization

```

i = 0;
if (i = 1)
{
    x = x + 5;
}

```

After optimization

```

i = 0;

```

Q. Optimize the code?

#include <iostream>

using namespace std;

int main()

{

int num;

num = 10;

cout << "Hello!";

return 0;

cout << num;

}

After elimination

int main()

{

int num;

num = 10;

cout << "Hello!";

return 0;

}

Date: _____

Day (M T W T F S)

Q. Optimize the code:

i = 1

while (i < 10)

{

y = i * 4;

}

After reduction

i = 1

t = 4;

while (t < 40)

{

y = t

t = t + 4;

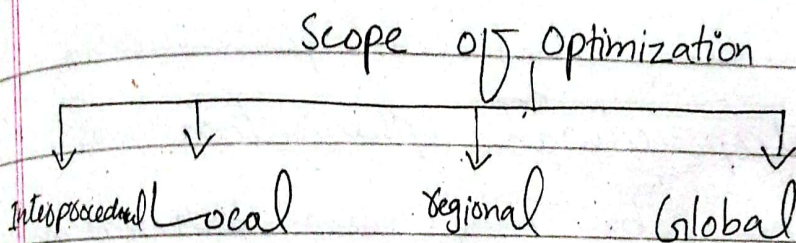
}

Q. Where to apply code optimization?

- o Source program
- o Intermediate code
- o Target code
- o Local optimization
- o Regional optimization
- o Global optimization
- o Interprocedural optimization

Scope of Optimization

"Area where we are doing optimization is known as scope of optimization."



Local vs Global

o Doing within the basic block.

Methods:

- o Local common subexpression elimination
- o Dead code elimination
- o Constant propagation
- o Algebraic law

o Between basic blocks.

Method:

- o Extended to entire control flow graph.

Q What is inter-procedural optimization?

- Interprocedural optimization refers to the optimization of whole programs which includes different procedures.
- The term interprocedural means between different procedures.
- Interprocedural optimization is done by using two techniques:
 - Inline substitution
 - Procedure placement
- It aims to improve the overall program performance by making global decisions about function calls, inlining code, and optimizing data and control flow across function boundaries.
- This approach enhances code efficiency and runtime performance. Spans multiple procedures & files. Time consuming.

Q What is Regional optimization?

- Regional optimization refers to optimization in an extended block.
- Also known as loop optimization.
- It involves analyzing and optimizing the code within loops.

Q. Write features of optimized codes

⇒ Optimized Code features:

- Execute faster
- Code size get reduced
- Efficient memory usage
- yielding better performance
- Reduces the time & space complexity

Machine dependent Optimization:

machine dependent optimization involves transformations that take into consideration, the properties of the target machine: like registers and special machine instruction sequences available.

⇒ These techniques are applied on generated target code.

Peephole Optimization

⇒ Peephole optimization is a simple and effective technique for locally improving target code.

⇒ This technique is applied to improve the performance of the target program by examining the short sequence of target instructions and replacing these instructions by shorter or faster sequence.

o small set of instructions or small
Date: ^{part of code to which optimization is performed}

[a peephole - a small window is moved over the target code and transformations can be made.]

o Peephole is a machine dependent technique.

Characteristics of peephole Optimization:

⇒ The peephole optimization can be applied on the target code using following characteristic:

- o Redundant instruction elimination
- o Flow of control optimization
- o Algebraic Simplification
- o Use of machine idioms

1) Redundant instruction elimination:

⇒ Especially the redundant loads and stores can be eliminated in this type of transformations:

for example:

MOV R₀, x

MOV x, R₀

Date: The second instruction can be eliminated because x is already in R₀.

(2) Flow of control optimization:

⇒ Using peephole optimization unnecessary jumps or jumps can be eliminated.

for example:

goto test	goto done
...	...
test: goto done	test: goto done
...	...
done:	done:

for example:

if a < b goto test	if a < b goto done
...	...
test: goto done	test: goto done
...	...
done:	done:

(3) Algebraic Simplification:

⇒ Peephole optimization is an effective technique for algebraic simplification.

⇒ The statement such as:

$$x = x + 0$$

$$\text{or } x = x * 1$$

can be eliminated by peephole optimization.

⇒ Because value of x does not change.

(4) Reduction in strength:

⇒ Certain machine instructions are cheaper than the others.

In order to improve the performance of the intermediate code we can replace these instructions by equivalent cheaper instructions.

⇒ for example, x^2 is cheaper than $x * x$.

⇒ Similarly addition and subtraction is cheaper than multiplication and division.

(9) Machine idioms:

○ The target instructions have equivalent machine instructions for performing some operations.

○ Hence, we can replace these target instructions by equivalent machine instructions in order to improve the efficiency.

○ $x = x + 1$ → Remove and use Auto-increment.

○ $x = x - 1$ → Remove and use Auto-decrement.

What is code optimization?

⇒ Optimization is a program transformation technique, which tries to improve the code to consume less resources and delivers high speed.

⇒ The output code must not change the meaning of the program in any way.

- Should not change the output produced for any input
- Should not introduce any error.

⇒ Optimization should increase the speed of the program and if possible, the program should demand less resources.

Optimization should itself be fast and should not delay the overall compilation process.

Q Optimize the following code using optimization rules.

$$a = x^2$$

$$b = 3$$

$$c = x$$

$$d = c * c$$

$$e = b * 2$$

$$f = a + d$$

$$g = e * f$$

Algebraic Optimization:

The statement $a = x^2$ is written as $a = x * x$

Copy propagation:

As $c = x$ can directly put x in place of c in statement

$d = c * c$ then it will be

$$d = x * x$$

Date:

Day: MTWTFSSConstant folding:

Then we can also directly put value of b in statement $e = b \times 2$. then $e = 3 \times 2$ will be $e = 6$.

Common Subexpression elimination:

As from algebraic optimization, we have $a = x \times x$, and we also have $d = x \times x$ from copy propagation, replace $d = a$.

Copy Propagation (sand a)

As $d = a$, we can place a in place of d in $f = a + d$, so

$$f = a + a$$

And already have value of $e = 6$ from Constant folding then $g = e \times f$ will be $g = 6 \times f$

Date:

Dead Code elimination

$$a = x \times x$$

$$b = 3$$

$$c = x$$

$$d = a$$

$$e = 6$$

$$f = a + a$$

$$g = 6 \times f$$

Here dead code let remove:

$$a = x \times x$$

$$f = a + a$$

$$g = 6 \times f$$

This is the final code after optimization.

Date: _____

Day: MTWTFSS

Q. Optimize the following code:

```
int j, x, y, z;
```

```
x = 0;
```

```
y = 1;
```

```
z = 1;
```

```
for (j = 0; j <= N; j++)
{
```

```
    x = (j+1) * (j+1) + (j+1) * x + (8 * a / b) * j;
```

```
    x = x + z * y;
```

```
}
```

Code movement:

- The statement $x = x + z * y$ can be done in the following way

```
temp2 = z * y
```

```
x = x + temp2
```

And put temp2 outside the loop.

Loop invariant:

$\Rightarrow$ The expression $8 * a / b$ can be avoided overhead. place it outside the loop.

```
temp1 = 8 * a / b
```

Common sub expression elimination:

$\Rightarrow$ The code $(j+1) * (j+1) * (j+1)$ can be replaced by:

```
3 * (j+1)
```

to reduce the calculation.

Optimize code is:

```
int j, x, y, z;
```

```
x = 0;
```

```
y = 1
```

```
z = 1
```

```
float temp1 = 8 * a / b;
```

```
int temp2 = z * y
```

```
for (j = 0; j <= N; j++)
{
```

```
    x = 3 * (j+1) * x + temp1 * j;
    x = x + temp2;
}
```


Date: _____

Day: MTWTFSS

Q Optimize the following code with title of the technique you use:

$$x = 1$$

$$y = a * b + 3$$

$$z = a * b + x + z + 2$$

$$x = 3$$

Constant folding:

In statement:

$$z = a * b + x + z + 2$$

we replace x by $x = 1$

$$z = a * b + 1 + z + 2$$

$$z = a * b + z + 3$$

$$z = a * b + 3 + z$$

Common sub expression elimination:

In statement:

$$z = a * b + 3 + z$$

And

$$y = a * b + 3 + z$$

Since same sub expression so we can remove

$$y = a * b + 3$$

$$z = y + z$$

Code optimize is:

$$x = 1$$

$$y = a * b + 3$$

$$z = y + z$$

$$x = 3$$

OR $z = a * b + 3$
Remain

Code optimize is:

$$x = 1$$

$$y = a * b + 3$$

$$z = y + z$$

$$x = 3$$

OR $x = 1$
 $y = a * b + 3$
 $z = a * b + 2 + 3$
 $x = 3$

Dead code elimination

After dead code elimination:

$$\boxed{y = a * b + 3}$$

$$z = y + z$$

OR

$$\boxed{z = a * b + 3 + z}$$